

Django 教學 17

服務運行與發布維護

第 25～27 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [16_部署設定與正式環境](#)。

- **第 25 章：服務程序與監控**
- **第 26 章：Migration、備份與復原**
- **第 27 章：CI 與發布流程**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 25 章

服務程序與監控

畫出服務請求路徑，取得 log 與 health check 的可觀察證據。

本章的操作環境與成果

部署練習使用 LearnBoard 或 LearnMart 副本；LearnJournal 僅作延伸示例。

完成成果： 畫出服務請求路徑，取得 log 與 health check 的可觀察證據。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

25-1 Production request path

常見架構：

```
graph TD; Internet -->|HTTPS| Proxy[Reverse proxy / load balancer]; Proxy --> Server[WSGI/ASGI server]; Server --> Django
```

Internet
↓ HTTPS
Reverse proxy / load balancer
↓
WSGI/ASGI server
↓
Django

每一層都有不同責任。

25-2 Reverse proxy 常做什麼？

- TLS termination
- domain / virtual host
- static file serving
- request size / timeout
- rate limit
- compression
- access log

Django 不需要自己做完所有網路層工作。

25-3 WSGI / ASGI server

Django 提供：

- `config/wsgi.py`
- `config/asgi.py`

Production 常交給 Gunicorn、uWSGI、Uvicorn/Daphne 或平台內建 server。

選擇取決於 sync/async、平台與運維方式，不是「哪個名字最潮」。

25-4 Process manager 要解決什麼？

- crash 後重啟
- 多 worker
- graceful shutdown
- boot on restart
- stdout/stderr 收集
- health check

systemd、container orchestrator、PaaS 都可以扮演這個角色。

25-5 Log 不只 `print()`

至少分：

- application log
- access log
- error log
- audit/security log（視需求）

Production log 應可搜尋、保留適當期間，且不要洩漏 secrets/password/token。

25-6 Django logging 心智模型

```
import logging
logger = logging.getLogger(__name__)

logger.info("article published", extra={...})
```

真正 production 可以把 handler 指到 stdout，再由平台集中收集。

25-7 可觀測性

至少知道三種訊號：

- logs：發生了什麼
- metrics：整體趨勢
- traces：一次 request 穿過哪些服務

本課先把 log、HTTP status、response time 建立起來，再延伸監控。

25-8 Health check

Health endpoint 不等於「首頁回 200」。

要思考：

- process alive
- app ready
- database 是否必要
- 外部 service 掛掉時是否也要判 unhealthy

過度嚴格的 health check 也可能造成重啟風暴。

25-9 本章檢核

1. reverse proxy 與 Django 各負責什麼？
2. process manager 為什麼比 `nohup runserver` 好？
3. 哪些資料不應出現在 log？
4. health check 為什麼不一定要檢查所有外部服務？

第 25 章實作與離堂檢核

任務： 畫出服務請求路徑，取得 log 與 health check 的可觀察證據。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： 部署與維運 [workbook](#)。手冊保留原章號，對照表列出本課位置。

第 26 章

Migration、備份與復原

在練習副本完成備份與還原，記錄資料恢復結果及所需時間。

本章的操作環境與成果

部署練習使用 LearnBoard 或 LearnMart 副本；LearnJournal 僅作延伸示例。

完成成果： 在練習副本完成備份與還原，記錄資料恢復結果及所需時間。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

26-1 先備回顧：資料模型與檔案

資料層教材 已建立本章需要的觀念：

- Model 描述資料；migration 記錄結構的變更。
- unique／constraint 加入前，既有資料也必須符合規則。
- ImageField 保存 storage 路徑參照；DB 備份不包含圖片內容。
- Admin 改資料與修改 schema 不同；後台操作也需權限與紀錄。

本章接著處理正式部署順序、備份及復原。

26-2 Migration 是版本化 schema 變更

部署前先確認：

```
uv run python manage.py makemigrations --check
uv run python manage.py showmigrations
```

不要在 production 現場才臨時產生 migration。

26-3 Deploy 順序要能相容

理想的 migration：

1. 新舊 code 短時間可以共存
2. 先新增 nullable/default 欄位
3. backfill data
4. code 切換讀新欄位
5. 最後再移除舊欄位

這是 expand → migrate → contract 思路。

26-4 Data migration 也會失敗

LearnJournal 03A 已有 `RunPython` ◦

Production 額外要想：

- 資料量多大？
- transaction 多久？
- 會鎖表嗎？
- 可以重跑嗎？
- rollback 真的可逆嗎？

26-5 Backup 的最低標準

你要能回答：

- 備份什麼？DB / media / config
- 多久一次？
- 保留多久？
- 放在哪裡？
- 誰能讀？
- 有沒有測過 restore？

「有 backup job」不等於「能恢復」。

26-6 RPO / RTO

兩個常用概念：

- RPO：最多能接受遺失多少時間的資料？
- RTO：服務最多能停多久？

這兩個答案決定 backup 頻率與 recovery 設計。

26-7 Rollback 不只是 git revert

Code rollback 容易；database rollback 可能困難。

例如：

- 新 code 已寫入新 schema
- migration 已刪欄位
- data migration 不可逆

所以 schema change 要設計向前/向後相容窗口。

26-8 Classroom recovery drill

SQLite 可以做最小演練：

```
cp db.sqlite3 db.sqlite3.backup  
# 做一個破壞性操作  
cp db.sqlite3.backup db.sqlite3
```

這只是概念演練；production PostgreSQL 應使用原生/managed backup 與 restore 流程。

26-9 本章檢核

1. `makemigrations` 應該在 production server 臨時跑嗎？
2. backup 成功紀錄為什麼不等於 restore 可用？
3. RPO / RTO 分別回答什麼？
4. 為什麼 database rollback 比 code rollback 難？

第 26 章實作與離堂檢核

任務： 在練習副本完成備份與還原，記錄資料恢復結果及所需時間。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： 部署與維運 `workbook`。手冊保留原章號，對照表列出本課位置。

第 27 章

CI 與發布流程

建立 CI 與發布清單，列出發布前後檢查、停止條件及復原方式。

本章的操作環境與成果

部署練習使用 LearnBoard 或 LearnMart 副本；LearnJournal 僅作延伸示例。

完成成果： 建立 CI 與發布清單，列出發布前後檢查、停止條件及復原方式。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

27-1 最小 CI gate

每個 Django 專案至少：

```
uv run python manage.py check
uv run python manage.py makemigrations --check
uv run python manage.py test
```

Repository 教材再加：

```
python scripts/check_material_links.py
```

27-2 為什麼 `makemigrations --check` ?

它抓這種情況：

```
models.py 已改  
migration 忘了 commit
```

沒有它，production migrate 可能「成功」但 schema 根本沒包含你的 model change。

27-3 CI 不應保存 production secrets

Test settings 使用：

- SQLite / disposable DB
- locmem email/cache
- fake credentials

若 integration test 必須用 secret，使用 CI secret store，不要寫進 YAML。

27-4 Release 前 checklist

- tests green
- migrations reviewed
- backup verified
- environment variables present
- `DEBUG=False`
- `ALLOWED_HOSTS` correct
- static collected
- health check ready
- rollback plan understood

Checklist 不是官僚文件，是把記憶外部化。

27-5 Release 後驗證

Smoke test :

- homepage 200
- login/logout
- 一個核心 CRUD flow
- static/media 可讀
- DB write 正常
- error log 沒爆量
- scheduled job 有在跑

27-6 監看 deployment

上線後短時間重點看：

- 5xx rate
- latency
- database errors
- login failures
- background job failures
- disk/storage usage

不要部署完就關 terminal 當作完成。

27-7 Roll forward vs rollback

小錯誤有時候最快是 hotfix roll forward；重大或未知風險才 rollback。

重點是事先知道：

- code 是否可 rollback
- migration 是否相容
- data 是否被不可逆修改

27-8 本章檢核

1. CI 最小三個 Django command 是什麼？
2. `makemigrations --check` 抓哪一類錯？
3. release checklist 為何有價值？
4. deploy 後為什麼還要 smoke test / monitor？

27-9 Production readiness 地圖

```
graph TD; A[Code + tests] --> B[Environment / secrets]; B --> C[Database + migrations]; C --> D[Static / media]; D --> E[WSGI/ASGI + proxy + HTTPS]; E --> F[Logs / health / monitoring]; F --> G[Backup / restore / scheduler];
```

Code + tests
↓
Environment / secrets
↓
Database + migrations
↓
Static / media
↓
WSGI/ASGI + proxy + HTTPS
↓
Logs / health / monitoring
↓
Backup / restore / scheduler

27-10 LearnBoard 與 LearnMart 的部署重點

專案	特別檢查
LearnBoard	帳號、操作頻率限制、稽核紀錄
LearnMart	media、庫存與交易、訂單隱私、付款邊界

LearnJournal 的 cache、email、scheduler 與 search 留作延伸閱讀。
本套課程的部署驗收不以完成 LearnJournal 為前提。

27-11 最後驗證

每個專案：

```
uv run python manage.py check
uv run python manage.py makemigrations --check
uv run python manage.py test
```

Repository：

```
python scripts/check_material_links.py
```

Production-like settings：

```
uv run python manage.py check --deploy
```

27-12 全課程完成檢核

- 從空目錄建立 Django 練習，說明 request 如何得到 response。
- 設計、查詢與管理資料，完成有狀態的搜尋與分頁。
- 驗證表單、身份與物件權限，追蹤交易成功及失敗的資料變化。
- 用測試守住規則，管理 migration 與正式環境設定。
- 說明服務、檔案與資料庫的責任，完成備份還原與發布檢核。

最後交付：可重現的操作說明、測試紀錄、部署檢核與還原證據。

第 27 章實作與離堂檢核

任務： 建立 CI 與發布清單，列出發布前後檢查、停止條件及復原方式。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： 部署與維運 `workbook`。手冊保留原章號，對照表列出本課位置。

本份完成與後續

全課完成：整理功能、測試、部署與還原證據。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。